```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math

print("OpenCV:", cv2.__version__)
print("Experiment 8 started successfully!")
```

```
OpenCV: 5.0.0
Experiment 8 started successfully!
```

```
WIDTH, HEIGHT = 640, 360
NUM_FRAMES = 60
frames = []

for i in range(NUM_FRAMES):
    frame = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)

    # Moving rectangle
    x = 30 + 6 * i
    y = 70 + int(15 * np.sin(i / 6))

    cv2.rectangle(frame, (x, y), (x + 70, y + 60),
                  (0, 220, 255), -1)

    # Corners for tracking
    cv2.circle(frame, (x + 18, y + 18), 6, (255, 255, 255), -1)
    cv2.circle(frame, (x + 52, y + 42), 6, (255, 255, 255), -1)
    cv2.line(frame, (x + 10, y + 45), (x + 60, y + 10),
             (0, 0, 0), 3)

    # Second moving object
    cx = 520 - 4 * i
    cy = 240 + int(25 * np.sin(i / 5))
    cv2.circle(frame, (cx, cy), 24, (255, 120, 20), -1)
    cv2.circle(frame, (cx, cy), 8, (255, 255, 255), -1)

    frames.append(frame)

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(frames[0], cv2.COLOR_BGR2RGB))
plt.title("Sample Video Frame")
plt.axis("off")
plt.show()

print("Total frames:", len(frames))
```




```
Total frames: 60
```

```
gray = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)

feature_params = dict(
    maxCorners=150,
    qualityLevel=0.01,
    minDistance=7,
    blockSize=7
)

corners = cv2.goodFeaturesToTrack(
    gray, mask=None, **feature_params
)

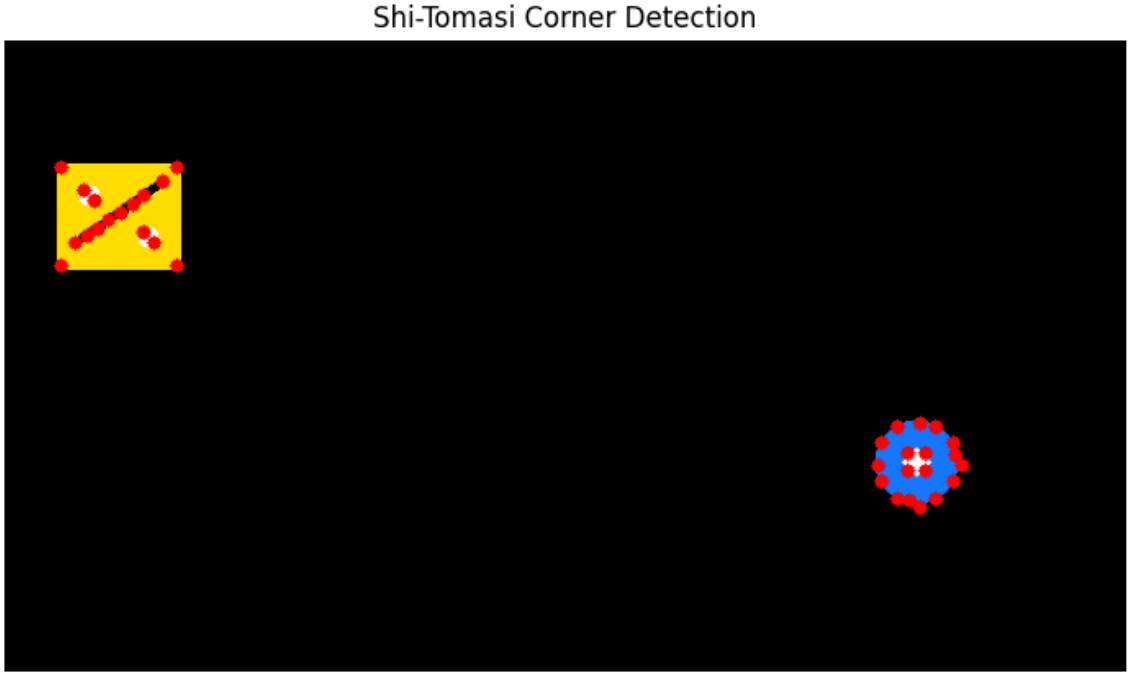
corner_image = frames[0].copy()

if corners is not None:
    for corner in corners:
        x, y = np.round(corner[0]).astype(int)
        cv2.circle(corner_image, (x, y), 4, (0, 0, 255), -1)

    print("Detected feature points:", len(corners))
else:
    print("No corners detected.")

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(corner_image, cv2.COLOR_BGR2RGB))
plt.title("Shi-Tomasi Corner Detection")
plt.axis("off")
plt.show()
```

```
Detected feature points: 34
```




```
old_gray = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray, mask=None, **feature_params
)

trail_mask = np.zeros_like(frames[0])
tracked_frames = []
displacements = []
tracked_counts = []

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10, 0.03
    )
)

for i in range(1, len(frames)):
    frame = frames[i].copy()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    if p0 is None or len(p0) == 0:
        p0 = cv2.goodFeaturesToTrack(
            old_gray, mask=None, **feature_params
        )

    if p0 is None:
        old_gray = gray.copy()
        continue

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray, gray, p0, None, **lk_params
    )

    if p1 is None or status is None:
        p0 = cv2.goodFeaturesToTrack(
            gray, mask=None, **feature_params
```

```
        )
        old_gray = gray.copy()
        continue

    good_new = p1[status.ravel() == 1]
    good_old = p0[status.ravel() == 1]

    step_distances = []

    for new, old in zip(good_new, good_old):
        nx, ny = np.round(new.reshape(-1)).astype(int)[:2]
        ox, oy = np.round(old.reshape(-1)).astype(int)[:2]

        cv2.line(
            trail_mask,
            (nx, ny),
            (ox, oy),
            (0, 255, 0),
            2
        )

        cv2.circle(
            frame,
            (nx, ny),
            4,
            (0, 0, 255),
            -1
        )


        new_point = new.reshape(-1)
        old_point = old.reshape(-1)


        dx = float(new_point[0] - old_point[0])

        dy = float(new_point[1] - old_point[1])

        step_distances.append(math.hypot(dx, dy))

    result = cv2.add(frame, trail_mask)
    tracked_frames.append(result)
    tracked_counts.append(len(good_new))

    displacements.append(
        float(np.mean(step_distances))
        if step_distances else 0.0
    )

    if len(good_new) >= 5:
        p0 = good_new.reshape(-1, 1, 2)
    else:
        p0 = cv2.goodFeaturesToTrack(
            gray, mask=None, **feature_params
        )

    old_gray = gray.copy()

print("Lucas-Kanade tracking completed.")
print("Processed frames:", len(tracked_frames))

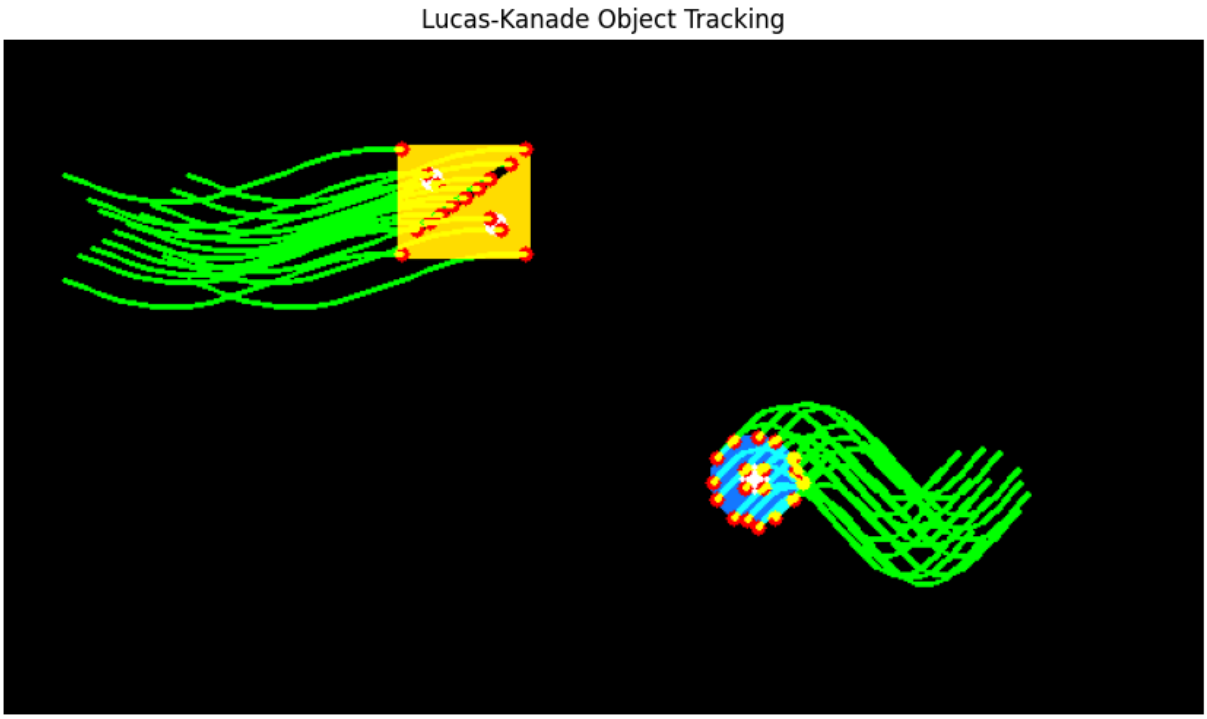
if tracked_frames:
    plt.figure(figsize=(12, 6))
    plt.imshow(
        cv2.cvtColor(
            tracked_frames[len(tracked_frames) // 2],
            cv2.COLOR_BGR2RGB
```

```
        )
    )
    plt.title("Lucas-Kanade Object Tracking")
    plt.axis("off")
    plt.show()

print(
    "Average tracked points:",
    round(float(np.mean(tracked_counts)), 2)
    if tracked_counts else 0
)

print(
    "Average displacement:",
    round(float(np.mean(displacements)), 3)
    if displacements else 0,
    "pixels/frame"
)
```

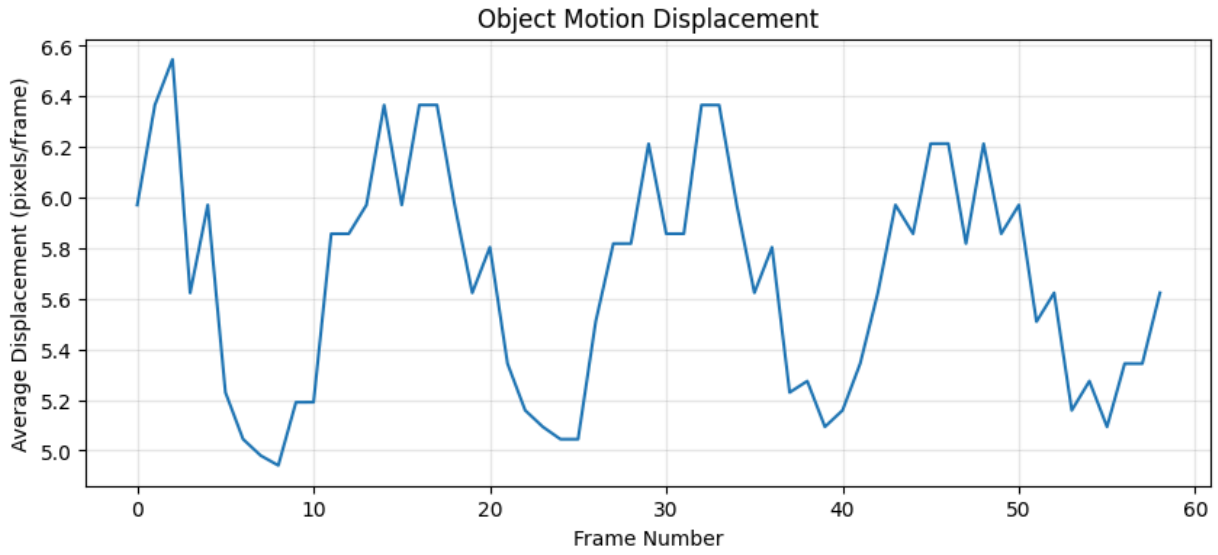
```
Lucas-Kanade tracking completed.
Processed frames: 59
```

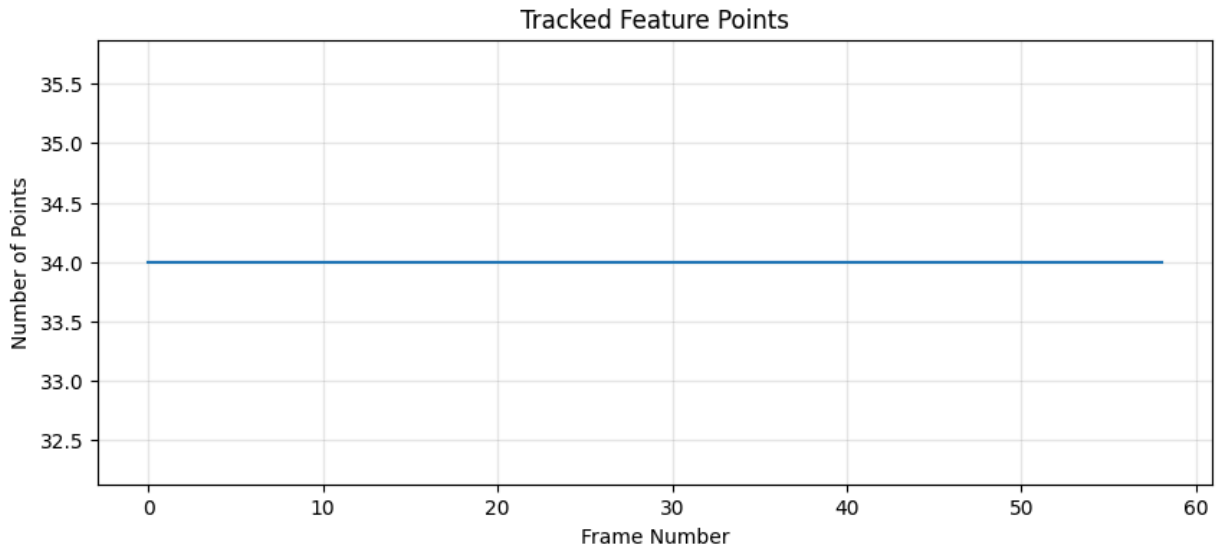



```
Average tracked points: 34.0
Average displacement: 5.676 pixels/frame
```

```
plt.figure(figsize=(10, 4))
plt.plot(displacements)
plt.title("Object Motion Displacement")
plt.xlabel("Frame Number")
plt.ylabel("Average Displacement (pixels/frame)")
plt.grid(True, alpha=0.3)
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(tracked_counts)
plt.title("Tracked Feature Points")
plt.xlabel("Frame Number")
plt.ylabel("Number of Points")
plt.grid(True, alpha=0.3)
plt.show()
```






```
prev_gray = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)

dense_images = []
mean_magnitudes = []

for i in range(1, len(frames)):
    next_gray = cv2.cvtColor(frames[i], cv2.COLOR_BGR2GRAY)

    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        next_gray,
        None,
        0.5, 3, 15, 3, 5, 1.2, 0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0], flow[..., 1]
    )

    hsv = np.zeros_like(frames[i])
    hsv[..., 1] = 255
    hsv[..., 0] = (
        angle * 180 / np.pi / 2
    ).astype(np.uint8)

    hsv[..., 2] = cv2.normalize(
```

```
            magnitude, None, 0, 255, cv2.NORM_MINMAX
        ).astype(np.uint8)

        flow_rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

        dense_images.append(flow_rgb)
        mean_magnitudes.append(float(np.mean(magnitude)))

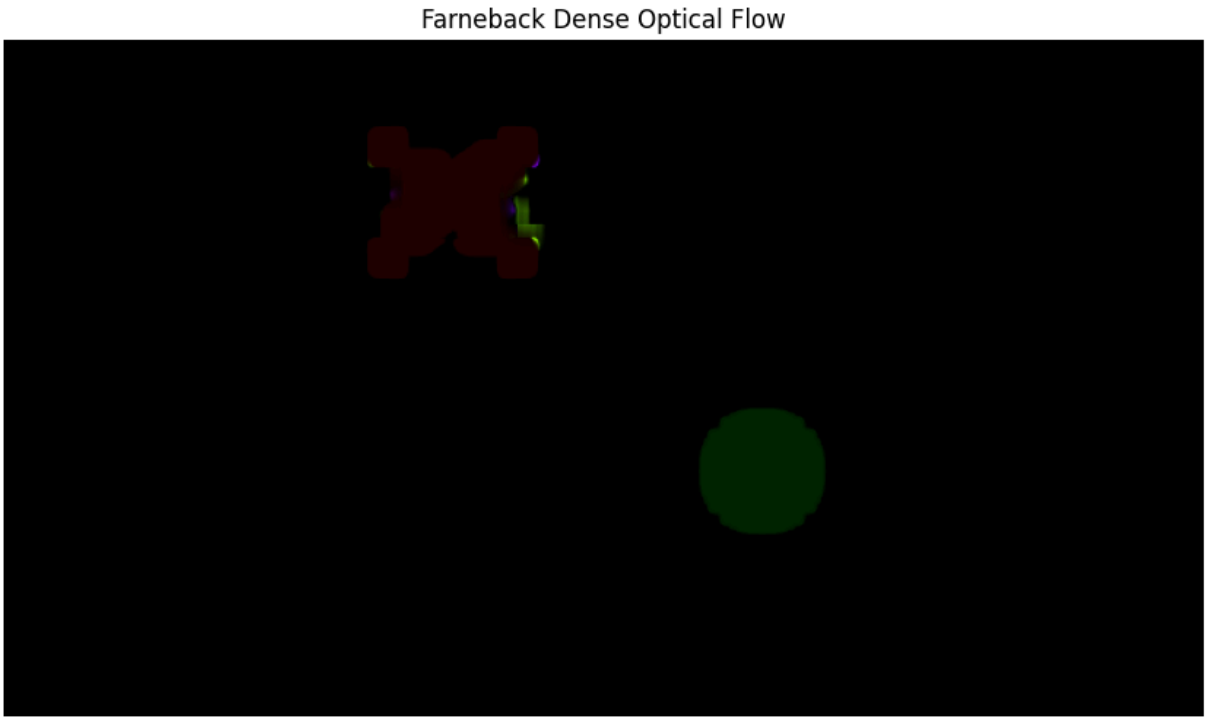
        prev_gray = next_gray

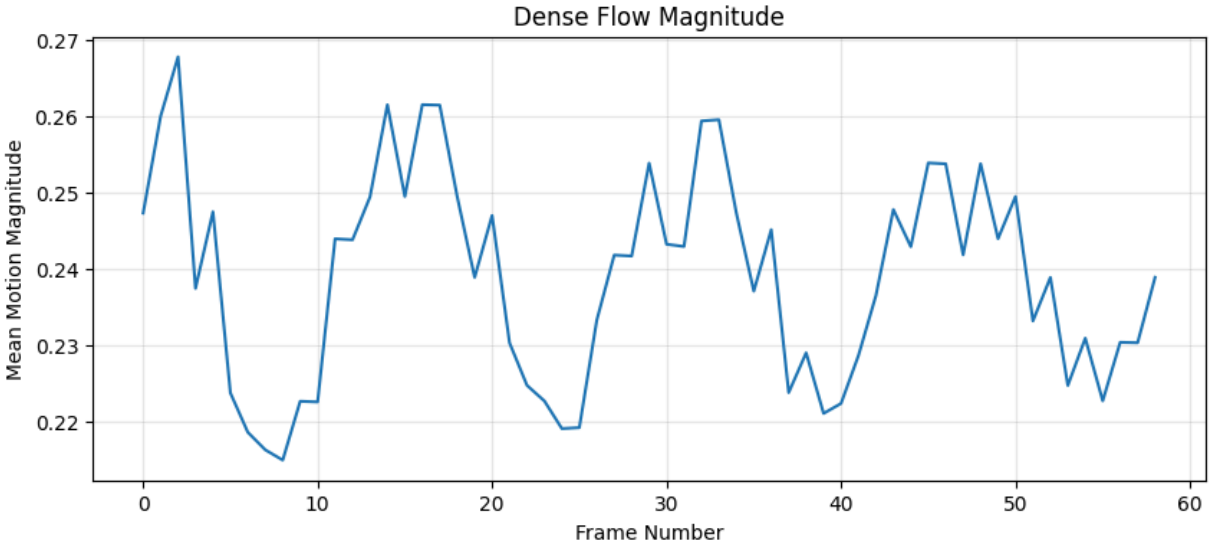
    print("Farneback Dense Optical Flow completed.")

    plt.figure(figsize=(12, 6))
    plt.imshow(dense_images[len(dense_images)//2])
    plt.title("Farneback Dense Optical Flow")
    plt.axis("off")
    plt.show()

    plt.figure(figsize=(10, 4))
    plt.plot(mean_magnitudes)
    plt.title("Dense Flow Magnitude")
    plt.xlabel("Frame Number")
    plt.ylabel("Mean Motion Magnitude")
    plt.grid(True, alpha=0.3)
    plt.show()
```

```
Farneback Dense Optical Flow completed.
```






```
print("OPTICAL FLOW COMPARISON")
print("-" * 40)
print("Lucas-Kanade: tracks selected feature points.")
print("Farneback: estimates motion across nearly every pixel.")
print("Lucas-Kanade: useful for tracking points and trajectories.")
print("Farneback: useful for analyzing overall scene movement.")
```

```
OPTICAL FLOW COMPARISON
----------------------------------------
Lucas-Kanade: tracks selected feature points.
Farneback: estimates motion across nearly every pixel.
Lucas-Kanade: useful for tracking points and trajectories.
Farneback: useful for analyzing overall scene movement.
```

```
answers = [
    (
        "Q1. How does object tracking differ from object detection?",
        "Object detection identifies objects in an image or video frame. "
        "Object tracking follows those objects across consecutive frames "
```

```
            "and estimates their movement over time."
        ),
        (
            "Q2. Explain how Optical Flow can be used for real-time object tracking.",
            "Optical Flow estimates the movement of image points between "
            "consecutive frames. Feature points are detected and tracked, "
            "and their motion vectors and trajectories are displayed."
        ),
        (
            "Q3. What is the role of Shi-Tomasi Corner Detection?",
            "Shi-Tomasi detects strong corners or feature points that can "
            "be tracked reliably. Lucas-Kanade estimates the displacement "
            "of these points between frames."
        ),
        (
            "Q4. Why is Optical Flow suitable for motion analysis in videos?",
            "It estimates pixel movement between frames and helps calculate "
            "motion direction, displacement, trajectories and movement patterns."
        ),
        (
            "Q5. What challenges arise while tracking fast-moving or partially occluded objects?"
            "Fast movement may cause large displacement and motion blur. "
            "Occlusion hides feature points. Lighting changes and low-texture "
            "regions can also reduce tracking accuracy."
        ),
        (
            "Q6. Compare Optical Flow-based tracking with deep learning-based tracking.",
            "Optical Flow estimates movement from consecutive frames and can "
            "track features without a trained object detector. Deep learning "
            "methods use trained models to detect or identify objects and "
            "often require more computational resources."
        ),
        (
            "Q7. How can Optical Flow be used in traffic monitoring and autonomous driving?",
            "It helps estimate the movement of vehicles, cyclists and "
            "pedestrians. This motion information can support traffic analysis "
            "and autonomous navigation when combined with other methods."
        ),
        (
            "Q8. Explain the effect of camera motion on Optical Flow estimation.",
            "Camera movement creates apparent motion across the frame, "
            "including the background. It becomes harder to distinguish "
            "camera-induced movement from independently moving objects."
        ),
        (
            "Q9. Mention five real-world applications of Optical Flow.",
            "1. Autonomous driving\n"
            "2. Video surveillance\n"
            "3. Robotics and navigation\n"
            "4. Human activity recognition\n"
            "5. Sports analytics"
        ),
        (
            "Q10. How can Optical Flow improve surveillance, robotics and human activity recognit
            "In surveillance, it helps detect moving regions. In robotics, "
            "it provides information about movement and nearby obstacles. "
            "In activity recognition, motion patterns help distinguish "
            "actions such as walking and running."
        )
    ]

    for question, answer in answers:
        print(question)
        print(answer)
        print("-" * 80)
```